REQ 4 Design Rationale:

Introduction:

In requirement 4, it requires the Inheritree to start from a "sprout" stage, grow into a "sapling", then a "young Inheritree", and finally a "mature Inheritree".

SOLID Principles:

Single Responsibility Principle:

SRP states that each class must only be made for one function. Hence, each class that we created is to show the different stages of the Inheritree growth. For example, the InheritreeSprout class is responsible for growing into an Inheritree Sapling after 3 ticks.

Open-Closed Principle:

OCP means that the design must be closed for modification but open for extension. This principle is achieved by making the FruitSpawner class as an Interface and the Tree class as an abstract class. Hence, if a new tree needs to be added in the future, a new class can be created without modifying the existing class.

Liskov Substitution Principle:

Each stage of the Inheritree is a subtype of the Tree class and can be used wherever a Tree is expected. This ensures that the program remains correct even if a subtype is used instead of the base type.

Interface Segregation Principle:

The FruitSpawner interface is implemented by the classes that can drop fruit. Classes that cannot drop fruit are not forced to implement this interface, avoiding unnecessary methods.

Dependency Inversion Principle:

The design does not explicitly show this principle, but it's a good practice to follow. High-level modules (like the game application) should not depend on low-level modules like specific tree stages. Both should depend on abstractions like the Tree class or FruitSpawner interface.

Pros:

- The code follows SOLID principles makes the code easier to maintain because changes in one part of the code are less likely to affect other codes
- The code has more reusability because of single responsibility principle is adhered

Cons:

- Following the SOLID principles may increase complexity of the code
- The interfaces might face duplicates that may violate DRY (Don’t repeat yourself), which abstract class does not violate DRY, for example, the drop fruit chance repeating itself violated DRY.